

in action, let us modify the key in 'tripleDES25.sage' from "RAINBOWS" to "RAINBOWX" (a change in just a single character) and execute the program. The new ciphertext obtained is: `fd794da244887abc8bc7c9073247e61158a40f99cd966f6b`. This ciphertext is completely different from the original: `fffb961a5099b22543c7adf47c8c5ee061d52ff30cc349be`. Despite changing just one character in the key, the output is unrecognisable, demonstrating strong confusion in the encryption process.

'Diffusion' ensures that every bit of plaintext spreads throughout the ciphertext, hiding statistical properties of the original message. A small change in the plaintext should affect multiple bits of the ciphertext—this is known as the avalanche effect. To observe diffusion, let us modify the plaintext from "HAPPINESSISABUTTERFLY" to "HAPPINESSISXBUTTERFLY", changing just one letter. Upon execution, the new ciphertext is: `fffb961a5099b22514035f3dcb702f4f61d52ff30cc349be`. Comparing it with the original ciphertext, we see that 16 characters in the middle have changed:

```
Original:  fffb961a5099b22543c7adf47c8c5ee061d52ff30cc349be
Modified:  fffb961a5099b22514035f3dcb702f4f61d52ff30cc349be
Changed Characters:  AAAAAAAAAAAAAAAAAA
```

This significant difference caused by a minor plaintext change highlights strong diffusion in the encryption process. If you ever design your own cryptographic algorithm—perhaps for an academic project—ensure that it incorporates both confusion and diffusion to enhance security.

The Advanced Encryption Standard (AES)

Though Triple DES replaced DES, it was slower due to its triple encryption process and remained vulnerable to certain cryptographic attacks, such as meet-in-the-middle attack. Because of these limitations, Triple DES was eventually replaced by the Advanced Encryption Standard (AES), which offers stronger security, better performance, and greater efficiency in both software and hardware implementations. Originally known as Rijndael, AES was developed by cryptographers Joan Daemen and Vincent Rijmen and was adopted by the US-based National Institute of Standards and Technology (NIST) in 2001. The National Centre for Communication Security (NCCS) in India also recommends AES.

Now, consider the SageMath program named 'aes26.sage' shown below, which implements AES using the package 'PyCryptodome'.

```
from Crypto.Cipher import AES
from Crypto.Random import get_random_bytes
import base64
```

```
def pad(text):
    return text + (16 - len(text) % 16) * chr(16 - len(text) % 16)

def unpad(text):
    return text[:-ord(text[-1])]

def encrypt_aes(plaintext, key):
    plaintext = pad(plaintext).encode('utf-8')
    cipher = AES.new(key, AES.MODE_CBC)
    ciphertext = cipher.encrypt(plaintext)
    return base64.b64encode(cipher.iv + ciphertext).
    decode('utf-8')

def decrypt_aes(ciphertext, key):
    data = base64.b64decode(ciphertext)
    iv = data[:16]
    encrypted_text = data[16:]
    cipher = AES.new(key, AES.MODE_CBC, iv)
    return unpad(cipher.decrypt(encrypted_text).
    decode('utf-8'))

key = get_random_bytes(16)
plaintext = "Open Source For you"
ciphertext = encrypt_aes(plaintext, key)
decrypted_text = decrypt_aes(ciphertext, key)
print("Plaintext: ", plaintext)
print("Encrypted Text: ", ciphertext)
print("Decrypted Text: ", decrypted_text)
```

A detailed discussion of the program is omitted due to space constraints. Also, a discussion of the workings of AES is beyond the scope of this series. Therefore, let us briefly explain the SageMath program 'aes26.sage'. This program implements AES in Cipher Block Chaining (CBC) mode. The function `pad()` ensures that the plaintext length is a multiple of 16 bytes by adding padding characters, while the function `unpad()` removes them after decryption. The function `encrypt_aes()` first pads the plaintext, then initialises an AES cipher object with a randomly generated 16-byte key, and encrypts the data using CBC mode. The initialisation vector (IV) and ciphertext are then base64-encoded for safe storage or transmission. The function `decrypt_aes()` reverses this process by extracting the IV, decrypting the ciphertext, and removing the padding to recover the original message. Finally, the program generates a random AES key, encrypts the string "Open Source For you", and then decrypts it, demonstrating that the encryption and decryption processes work correctly. Note that because a random key is used each time, the generated ciphertext will be different with each execution of the program. Upon execution of the program 'aes26.sage', the output shown in Figure 2 will be displayed on the terminal.